

SWASH: A Flexible Communication Framework with Sliding Window-Based Cache Sharing for Scalable DGNN Training

Abstract

Dynamic Graph Neural Networks (DGNNs) are effective at capturing multidimensional data and enable many important applications. As model training is computationally intensive, distributed DGNN training is employed to accommodate large data. Also, when training DGNNs, so-called sliding window training is used predominantly, as it enhances both accuracy and efficiency. However, current distributed frameworks—such as snapshot partitioning, chunk-based partitioning, and L -hop cache-based communication-free vertex partitioning—are inherently incompatible with sliding window training. Although communication-based vertex partitioning is compatible with sliding window training, it is originally designed for static graphs, which limits the effectiveness of strategies like partitioning and embedding caching in distributed DGNN training. To support distributed sliding window training, we present SWASH, a scalable and flexible communication framework that utilizes a **Sliding Window-based cAche SHaring** technique. Specifically, we propose a flexible communication framework that supports communication ratio adjustment and communication timing selection, as well as hyperparameter settings and adaptive scheduling. We also propose a lightweight partitioning strategy tailored to sliding window-based DGNN training to reduce both partitioning and communication overheads. Finally, to alleviate decreases in accuracy due to reduced communication, we propose a cache-sharing technique based on sliding windows for sharing boundary vertex embeddings. Comprehensive experiments show that SWASH is capable of training speedups of an average of $9.44\times$ over state-of-the-art frameworks while maintaining the accuracy of fully communicating, non-caching training frameworks.

CCS Concepts

• **Computing methodologies** → **Distributed computing methodologies**; • **Information systems** → **Data management systems**.

Keywords

distributed training, dynamic graph neural networks, cache, sliding windows, partitioning strategies, communication frameworks

ACM Reference Format:

. 2025. SWASH: A Flexible Communication Framework with Sliding Window-Based Cache Sharing for Scalable DGNN Training. In *Proceedings of SIGMOD*. ACM, New York, NY, USA, 14 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD, June 03–05, 2025, Berlin, Germany

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Dynamic Graph Neural Networks (DGNNs) have achieved remarkable success in many applications, including in traffic flow forecasting [21, 48], epidemic prediction [24, 47], and social network analysis [7, 8, 13]. This success is enabled by their ability to capture graph attributes, structure, and evolution. However, training DGNNs on large-scale dynamic graphs incurs high computational costs and does not scale well to rapidly evolving Graph Neural Networks (GNNs). Specifically, DGNNs use Recurrent Neural Networks (RNNs) to model temporal dynamics by establishing a learning sequence across snapshots, tracking changes in vertex embeddings over time. This approach typically incurs communication and computation overheads that are T times higher than those of traditional GNNs, where T is the number of snapshots.

The two primary bottlenecks in distributed DGNN training are communication and synchronization. Communication includes i) *GNN communication*, where embeddings are collected from neighbors within the same snapshot, and ii) *RNN communication*, where hidden states are transferred across different snapshots. Synchronization challenges involve i) *snapshot synchronization*, requiring computations for snapshot $t + 1$ to wait for those for snapshot t to finish; ii) *layer synchronization*, where layer $l + 1$ computations depend on layer l computations; and iii) *iteration synchronization*, which mandates that iteration $i + 1$ must wait for iteration i to complete.

So-called *sliding window*-based DGNN training [12, 25, 26, 38] uses N_s snapshots for training in each iteration instead of using full snapshots of length T . It reduces complexity from being proportional to the number of snapshots T to N_s , where $N_s < T$. This mode reduces computational overheads, avoids the impact of stochastic gradients on convergence, and introduces sample diversity, enhancing efficiency and generalization. However, training on large dynamic graphs often exceeds a single machine's processing capacity, even with sliding windows. Distributed training frameworks offer a promising solution for training on large dynamic graphs. However, distributed DGNN training still faces difficult challenges. **Challenge I: Existing distributed frameworks require full communication, incurring substantial communication overheads.** Current distributed DGNN training relies on full communication, necessitating the transfer of embeddings for all vertices in each iteration. While this framework ensures accurate computations, it is highly inefficient. Relaxing communication constraints may not impact convergence rates or final accuracy significantly; randomness resulting from reduced communication may even enhance model generalization. Thus, it is pertinent to explore more flexible communication frameworks for distributed DGNN training.

Challenge II: Existing partitioning strategies fail to support efficient sliding window DGNN training. Many static graph partitioning strategies [2, 11, 14, 15, 37] poorly adapt to dynamic graphs. Also, minimizing communication within individual snapshots does not guarantee minimal communication across all snapshots. Further, while concatenating dynamic graphs into a single

large graph enables the use of static partitioning techniques, it incurs significant overheads. Next, current dynamic graph partitioning strategies [5, 33] focus primarily on full-snapshot DGNN training, making them unsuitable for sliding window training. As a result, a lightweight and effective partitioning strategy tailored to sliding window-based DGNN training is needed urgently.

Challenge III: Existing caching techniques do not reduce communication during sliding window training, causing communication bottlenecks. Existing caching techniques [12, 17] for DGNN training improve efficiency by storing vertex aggregation results for reuse when performing computations on subsequent snapshots, reducing redundant computations. However, they do not address the high communication overhead in distributed scenarios. While some caching techniques [27] for GNN training reduce network communication by reusing stale messages, they disregard caching between sliding windows, as distributed GNN training does not support sliding windows. However, the use of sliding windows presents an important yet unexplored optimization opportunity: sharing embedding messages between sliding windows to balance data staleness and training efficiency.

To tackle these challenges, we propose a flexible communication framework that utilizes sliding window based cache sharing. This framework supports the communication of vertex embeddings at any ratio at any time. We provide a set of empirical hyperparameter settings and propose an adaptive scheduling strategy to address **Challenge I**. We introduce a lightweight partitioning technique tailored to sliding window training to reduce communication and partitioning time, addressing **Challenge II**. Further, we propose a cache sharing technique that stores vertex embeddings from the snapshots within a sliding window and shares them across windows, reducing communication and alleviating accuracy degradation due to missing neighbor information, thus addressing **Challenge III**. Our contributions are as follows.

- We propose a flexible framework enabling communication ratio adjustment and timing selection with empirical hyperparameter settings, and an adaptive scheduling strategy to reduce communication overheads and mitigate accuracy degradation.
- We propose a lightweight partitioning strategy for distributed DGNN training that considers snapshot graphs, edge lifespans, and the effects of sliding windows, facilitating efficient partitioning through weight transformation and snapshot pruning.
- We propose a sliding window based cache sharing technique that aligns embedding sharing with snapshot positions, reducing communication frequency while preserving information and enhancing distributed DGNN training performance.
- We evaluate two common DGNN models on six datasets in two environments. Experimental results show that SWASH is capable of accelerating training by an average of 9.44× compared to state-of-the-art sliding window frameworks, without compromising the accuracy of fully communicating, non-cached frameworks.

The paper is organized as follows. Section 2 reviews related work on DGNN models, frameworks, and optimization techniques. Section 3 covers preliminaries on dynamic graphs and DGNN training. Section 4 details the SWASH system framework and partitioning strategies, while Section 5 covers the flexible communication framework, cache-sharing technique, and scheduling strategy of SWASH.

Section 6 provides theoretical analysis of the proposed techniques, and Section 7 reports on experimental studies. Finally, Section 8 concludes the paper.

2 Related Work

2.1 Dynamic Graph Neural Network Models

In real-world applications, dynamic graph data is often collected as snapshots at regular intervals. For example, social media platforms log user activity daily, city traffic sensors record vehicle counts every minute, and weather stations gather hourly meteorological data, all of which are organized as a sequence of graph snapshots. A number of neural network models [1, 6, 19, 25, 28, 30, 34, 45, 49, 54] target the effective handling of the resulting snapshot-based dynamic graphs, achieving state-of-the-art performance across different applications. SEIGN [28] introduces a dual evolution technique that evolves the parameters and embeddings of DGNNs simultaneously, customizing GNNs for each snapshot to capture inherent changes. ROLAND [45] adapts GNN architectures designed for static graphs to dynamic contexts while maintaining effectiveness. WinGNN [54] features a novel random gradient aggregation mechanism that reduces parameter size and enhances robustness. Additionally, a class of memory-based DGNNs [16, 40, 41, 44] target continuous-time dynamic graphs. The settings and computational processes they assume differ considerably from those of snapshot-based models. As a result, the systems [4, 18, 20, 43, 46, 51–53] that support them cannot accommodate snapshot-based DGNN training. Therefore, we exclude these systems from the scope of this paper. Several surveys [9, 22, 36, 50] highlight this distinction.

2.2 System Frameworks for DGNN Training

Training on a Single Server. Several single-server systems [17, 29, 38, 42] have been developed to enhance DGNN training efficiency. PiPAD [38] introduces a pipeline-parallel architecture with overlap-aware data organization to reduce transfer overheads, while PyGT [29] offers a unified library for DGNN training. However, their scalability is limited to single-server resources, making them unsuitable for large-scale dynamic graph training.

Training on Distributed Clusters. Table 1 summarizes the frameworks of existing distributed DGNN training systems, categorized into snapshot-based, vertex-based, and chunk-based partitioning systems. Here, Partition (e.g., PGC) refers to a specific partitioning strategy applied within a particular partitioning framework (e.g., the Chunk based Partitioning Framework). We outline their limitations and justify our choice of vertex partitioning.

(i) **Vertex partitioning based systems.** These extend distributed GNN systems [23, 27, 31, 32, 35, 37, 39], where each worker stores a subset of vertices from across all snapshots. While GNNs require communication to aggregate spatial neighbor embeddings, temporal neighbor information for RNNs remains complete. The quality of partitioning strategies impacts the communication efficiency in dynamic graphs. However, distributed GNN training does not incorporate sliding windows. Consequently, vertex partitioning encounters communication and synchronization bottlenecks when applied to distributed DGNN training. DynaHB [33] reduces communication by caching neighbors within L hops, but as DGNN layers increase, storage costs grow exponentially.

Table 1: State-of-the-art systems for DGNN training. Comm., Scala., and Appl. denote communication, scalability, and applicability, and ✓, ◊, and × denote good, moderate, and poor effectiveness, respectively.

System	Framework	Communication Schema	Partition	Cache	Comm.	Scala.	Appl.
CacheG [17]	Single	N/A	N/A	Combinations	N/A	×	◊
PiPAD [38]	Single	N/A	N/A	Aggregations	N/A	×	◊
PyGT [29]	Single	N/A	N/A	N/A	N/A	×	◊
ESDG [3]	Snapshot Partition	RNN	Range	N/A	×	◊	×
BLAD [10]	Snapshot Partition	N/A (lost sliding window)	Range	N/A	✓	✓	×
DGC [5]	Chunk-based Partition	GNN + RNN	PGC	N/A	◊	◊	×
DynaGraph [12]	Sequence-based Partition	Repartition	Random	Aggregations	×	✓	×
DynaHB [33]	Cached Vertex Partition	N/A (increased memory)	LAP	L-hop Features	✓	✓	◊
SWASH	Flex-Comm Vertex Partition	Partial GNN at Partial Epoch	LightMetis	Embeddings	✓	✓	✓

(ii) **Snapshot partitioning based systems.** These systems [3, 10, 12] partition snapshots across workers, eliminating GNN communication but still require temporal neighbor access across snapshots. ESDG [3] is a typical snapshot-based system. Yet, it struggles with sliding window training, as it is originally designed for full-snapshot training. BLAD [10] and DynaGraph [12] use snapshot partitioning to support sliding window training and require the data to be stored locally during training. Since each worker only stores part of the snapshots, cross-machine sliding window training cannot be performed. Further, as the sliding window length increases, the efficiency of snapshot partitioning in supporting the sliding window training mode decreases.

(iii) **Chunk partitioning based systems.** DGC [5] reduces communication in dynamic graphs by integrating structural and temporal neighbors into a single graph, utilizing Metis [15] for partitioning. However, this strategy is suited mainly for full-snapshot training and does not reduce communication for sliding windows effectively. Additionally, both RNNs and GNNs in these frameworks require communication, leading to two synchronization rounds and substantial overhead during sliding window training.

Justification for Our Choice. We propose a flexible communication vertex partitioning framework that offers several advantages over snapshot and chunk-based partitioning: (i) vertex partitioning aligns with the geographical distribution of data, e.g., subsidiaries within a group may retain all historical information about their employees; (ii) it enables more flexible communication operations; and (iii) it adapts seamlessly to sliding window training. A case study illustrates the flexibility of vertex partitioning in communication.

EXAMPLE 1. As shown in Figure 1, Vertex Partitioning (VP) splits snapshots between the first worker (wk_1) and the second worker (wk_2). Chunk Partitioning (CP) divides the graph into chunks for wk_1 and wk_2 , to reduce communication overheads. Snapshot Partitioning (SP) assigns the first two snapshots to wk_1 and the last two to wk_2 . In the lower left corner, SP and CP require transmitting the hidden state of each vertex from the current snapshot to the next snapshot via network communication for RNN computations. Next, VP requires communication at each snapshot to obtain the neighbor embeddings of vertices within that snapshot for GNN computations. Eliminating communication edges causes SP and CP to lose temporal neighbor information, making RNN computation impossible. In contrast, although VP loses some structural neighbor information, resulting in approximate vertex embeddings, it retains the complete computational logic. Thus, VP offers better flexibility than SP and CP when optimizing communication efficiency.

figure/case_study-eps-converted-to.pdf

Figure 1: Demonstration of vertex partitioning flexibility. Blue arrows represent RNN computation edges, red arrows represent GNN computation edges, and the dashed lines indicate communication edges. Green dashed lines partition the dynamic graph across workers. Blue circles denote target vertices. To simplify the figure, the arrowheads of all directed graph edges not related to target vertices are omitted.

2.3 Optimizations for DGNN Training

Partitioning Strategies. We first consider vertex partitioning. DynaHB [33] introduces a load-aware partitioning strategy, aimed at load balancing rather than communication efficiency, for communication avoidance. Additionally, several other GNN partitioning strategies [2, 11, 14, 15, 37] are ill-suited for distributed DGNN training, as they disregard temporal information. Many snapshot and chunk partitioning strategies [3, 5, 10, 12] also conflict with our vertex partitioning framework. Our LightMetis strategy aims to reduce total communication across sliding windows, while incorporating techniques to reduce data partitioning costs.

Caching Techniques. Various caching techniques have been proposed. CacheG [17] eliminates redundant computations by caching specific results for targets needing the same computations. PiPAD [38] and DynaGraph [12] cache aggregation results, as RNNs often reuse these constant results across different gates. However, these techniques reduce mainly the computational overheads on a single machine without effectively addressing communication costs. DynaHB [33] caches the initial features of L -hop neighbors to avoid communication, but it incurs high storage costs and redundant computations to cache all neighbors within L hops for each snapshot. SANCUS [27] focuses on caching intermediate embeddings and shares within a single static graph, but it lacks support for sliding window training. Consequently, it fails to leverage the

potential optimizations that could arise from sliding window-based cache sharing. In contrast, our proposed caching technique utilizes sliding windows to enhance sharing of intermediate embeddings.

3 Preliminaries

We define dynamic graphs, consistent with prior studies [5, 33, 38]; sliding windows; and the notions of structural and temporal neighbors. We end by specifying the distributed DGNN training.

DEFINITION 1 (DYNAMIC GRAPH). A dynamic graph is a sequence of directed graph snapshots, denoted as $\mathcal{G} = \langle \mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_T \rangle$. Each snapshot $\mathcal{G}_t$ is a four-tuple $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t, \mathcal{X}_{\mathcal{V}_t}, \mathcal{X}_{\mathcal{E}_t})$, where $\mathcal{E}_t$ and $\mathcal{V}_t$ are the edge and vertex sets at time t , respectively, and sets $\mathcal{X}_{\mathcal{V}_t}$ and $\mathcal{X}_{\mathcal{E}_t}$ denote the feature sets of the vertices and edges at time t , respectively. Here, features refer to vectors that represent the attribute characteristics of vertices or edges.

DEFINITION 2 (SLIDING WINDOW). We define a sliding window S_i in DGNN training as a subsequence of a dynamic graph $\mathcal{G}$ consisting of N_s consecutive snapshots, i.e., $S_i = \langle \mathcal{G}_i, \mathcal{G}_{i+1}, \dots, \mathcal{G}_{i+N_s-1} \rangle$.

For a dynamic graph $\mathcal{G}$ with T snapshots, the total number of sliding window samples is $T - N_s + 1$.

DEFINITION 3 (STRUCTURAL AND TEMPORAL NEIGHBORS). Let v and u be vertices in a snapshot $\mathcal{G}_t$ at time t in a dynamic graph $\mathcal{G}$. If there is an edge from v and u , denoted as $e_{u,v} \in \mathcal{E}_t$, u is referred to as a **structural neighbor** of v . The set of all structural neighbors of a vertex v is denoted as $N(v)$. Furthermore, the set of **temporal neighbors** of a vertex v , denoted as $\mathcal{T}(v)$, is the set of the same vertices v_t in different snapshots, where $t \in \{1, \dots, T\}$. In sliding window training, the set of temporal neighbors $\mathcal{T}(v)$ is restricted to the snapshot in the given sliding window.

Distributed DGNN Training. All sliding windows are processed in an epoch, yielding $T - N_s + 1$ iterations. Each iteration involves N_s snapshots, and the processing of a snapshot entails communication with the GNN according to Equation 1 and transforming the results through an RNN to generate hidden states for the subsequent snapshot according to Equation 2. According to this process, all snapshots in each sliding window must communicate with remote vertices and must synchronize.

$$\mathbf{h}_{v,t}^\ell = \text{GNN} \left(\text{AGG} \left(\mathbf{h}_{u,t}^{\ell-1} \mid u \in N(v_t) \right), \mathbf{h}_{v,t}^{\ell-1} \right) \quad (1)$$

Here, $\mathbf{h}_{v,t}^\ell$ is the representation of vertex v at time step t and layer ℓ . $N(v_t)$ is the set of structural neighbors of vertex v_t , and AGG is the aggregation function.

$$\mathbf{z}_{v,t}^\ell = \text{RNN} \left(\mathbf{h}_{v,t}^\ell, \mathbf{z}_{v,t-1}^\ell \mid u \in \mathcal{T}(v) \right) \quad (2)$$

Here, $\mathbf{z}_{v,t}^\ell$ is the hidden state of node v at time step t and layer ℓ , and $\mathcal{T}(v)$ is the set of temporal neighbors of vertex v .

4 System Framework and Partitioning

4.1 System Overview

Figure 2 provides an overview of SWASH, showing its five core system modules and their interconnections.

- **Lightweight Partitioning Module:** performs vertex partitioning, supporting a range of customizable partitioning strategies. Moreover, it provides a built-in lightweight partitioning strategy.
- **Encoding Module:** efficiently encodes sparse global indices of vertices and edges into compact, continuous local indices.

- **Communication Module:** manages message-passing operations for forward and backward propagation, optimizing boundary sampling according to predefined communication ratios, i.e., the ratios of boundary vertices communicated for each snapshot.
- **Caching Module:** responsible for managing the lifecycle of the sliding window based caching, overseeing the creation, access, and updating of cache data across iterations.
- **Scheduling Management Module:** manages the communication, cache updates, and cache utilization, ensuring efficient coordination throughout the training process.

A dynamic graph is initially stored in a distributed shared network file system (NFS). During the preprocessing stage of the training, each worker reads the data, and the lightweight partitioning module partitions the dynamic graph, generating a table that assigns vertex IDs contained in P_m to workers (Step ①). After receiving the partitioning results, workers load their local datasets from the NFS (Step ②). Each worker then encodes its local dataset, creating a mapping from global to local vertex and edge indices for training (Step ③) and constructing the cache on GPUs (Step ④).

During training, a trainable sliding window is used for forward propagation by the trainer (Step ⑤). For GNN aggregation, vertices require access to remote neighbors (referred to as boundaries). The scheduling module then chooses either communication-based or cache-based training (Step ⑥). In the communication-based training, it invokes the communication module to sample boundaries, which reduces the communication time based on the boundary sampling ratios. SWASH updates the cache with these boundaries during training. In the cache-based mode, the cache module bypasses communication and returns cached boundaries directly.

4.2 A Lightweight Partitioning Strategy

Communication is the main bottleneck in distributed DGNN training. While static graph partitioning strategies reduce communication overheads effectively but consider only the edges of a single snapshot, resulting in suboptimal performance when applied to distributed DGNN training. To address this, we propose a lightweight partitioning strategy tailored to distributed DGNN training.

The same vertex in different snapshots is assigned to the same worker to eliminate RNN communication. For example, if vertex v_1 is assigned to worker wk_m , the instances of v_1 in all snapshots are assigned to worker wk_m , ensuring that $\cup_{t=1}^T v_{1,t} \subseteq P_m$. Thus, the goal is to partition N vertices as in a static graph. In distributed DGNN training, partitioning must account for the edges from all snapshots to ensure accurate statistical analyses. We observe that the edges of initial dynamic graphs do not accurately reflect the actual communication edges during training. Aspects such as edge lifespans and sliding windows result in varying communication frequencies for edges across different snapshots. To clarify the proposed strategy, we further define the notions of raw snapshots, snapshots and edge-life transformation.

DEFINITION 4 (RAW SNAPSHOT, SNAPSHOT AND EDGE LIFE TRANSFORMATION). Continuing Definition 1, we define the raw dynamic graph as $\mathcal{R} = \langle \mathcal{R}_1, \mathcal{R}_2, \dots, \mathcal{R}_R \rangle$, where the **raw snapshot** $\mathcal{R}_r = (\mathcal{V}_r, \check{\mathcal{E}}_r, \mathcal{X}_{\check{\mathcal{V}}_r}, \mathcal{X}_{\check{\mathcal{E}}_r})$. **Edge life transformation** preserves edges from each snapshot to the subsequent z snapshots. **Snapshot** is generated by an edge-life techniques [3, 26, 33, 38], i.e., $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t, \mathcal{X}_{\mathcal{V}_t}, \mathcal{X}_{\mathcal{E}_t})$, where $\mathcal{V}_t = \cup_{r=t}^{t+z} \check{\mathcal{V}}_r$, $\mathcal{E}_t = \cup_{r=t}^{t+z} \check{\mathcal{E}}_r$.

figure/system_overview-eps-converted-to.pdf

Figure 2: System overview of SWASH. Red-highlighted modules indicate the innovations.

Each snapshot comprises multiple raw snapshots, and each sliding window contains several snapshots. To determine the usage frequency of each raw snapshot in training, we propose a two-level hierarchical strategy. Example 2 illustrates this process.

EXAMPLE 2. *With a sliding window length of 3, the frequencies of snapshots $\mathcal{G}_1, \mathcal{G}_2, \mathcal{G}_3, \mathcal{G}_4$, and $\mathcal{G}_5$ are 1, 2, 3, 2, and 1, respectively. Similarly, $\mathcal{G}_i$ is derived by merging the edges of multiple raw snapshots using the edge-life technique. With an edge-life length of 2, $\mathcal{G}_1$ uses the raw snapshots $\mathcal{R}_1$ and $\mathcal{R}_2$, while $\mathcal{G}_2$ uses the raw snapshots $\mathcal{R}_2$ and $\mathcal{R}_3$. In this case, when training with $\mathcal{G}_1$ and $\mathcal{G}_2$, the edges from $\mathcal{R}_1$ and $\mathcal{R}_3$ are used once, while the edges from $\mathcal{R}_2$ are used twice. We need to consider the combined effects of the sliding window and edge-life techniques to determine the usage counts for the raw dynamic graph $\mathcal{R}$. Notably, the number of edges in $\mathcal{R}$ is significantly smaller than that in $\mathcal{G}$. The edges in $\mathcal{R}$ are used as edge data, with edge frequencies serving as weights for weighted graph partitioning.*

Next, we outline the process of the computation. During training, the usage frequency of each snapshot varies, as does the frequency of each raw snapshot. Thus, two transformations are necessary: (i) to determine the usage count of snapshot $\mathcal{G}_t$ by sliding windows $\mathcal{S}$, and (ii) to determine the usage count of raw snapshot $\mathcal{R}_r$ by dynamic graphs $\mathcal{G}$. Specifically, the usage count UG_t for snapshot $\mathcal{G}_t$ is calculated as $UG_t = \min(t, T - t, N_s)$, while the usage count UR_r for raw snapshot $\mathcal{R}_r$ is given by $UR_r = \min(r, R - r, z)$. Specifically, to accurately evaluate the overall usage of raw snapshots, we first compute $U_{t,r}$, the usage count of each $\mathcal{G}_t$ for $\mathcal{R}_r$, and then aggregate these counts across all snapshots: $AR_r = \sum_{t=1}^T U_{t,r}$.

Having determined the usage count for each raw snapshot, we can associate these counts with the corresponding edges of raw snapshots. We need to sum the frequency of the same edge across different raw snapshots, which are as edge weights for partitioning. For example, if $\mathcal{R}_1$ is used 4 times and $\mathcal{R}_2$ is used 10 times, and edge (v, u) appears in both $\mathcal{R}_1$ and $\mathcal{R}_2$, the total usage count of edge (v, u) would be 14. This facilitates the construction of an individual large graph, which can subsequently be partitioned using strategies such as Metis [15] to reduce the communication overhead. However, the raw dynamic graphs still contain a substantial number of edges, making the computation of edge frequency accumulation resource-intensive. To address this, we further optimize the partitioning process to improve overall efficiency.

It turns out that a set of raw snapshots share the maximum usage count, referred to as $\mathcal{R}_{max}$. The counts for other raw snapshots decrease progressively in a non-linear fashion due to the two-level hierarchical strategy. Specifically, the decline is gradual

in the middle, but becomes much steeper towards the sides. By carefully selecting the left and right sides and retaining the intermediate snapshots, we can focus on the group with the highest and most consistent usage counts, optimizing the overall process.

From this observation, we propose a threshold r in Equation. 3, which retains raw snapshots with usage counts exceeding r .

$$r \geq \frac{N_{rmax}}{N_r} \cdot r_{max} = \frac{N_s \cdot z \cdot (|\mathcal{R}| - 2z - 2N_s + 4)}{|\mathcal{R}|}, \quad (3)$$

where N_{rmax} denotes raw snapshot number with the highest usage count, N_r is the number of raw snapshots, and r_{max} is the maximum usage count. Our technique prioritizes snapshots with the high usage counts. When N_{rmax} is large, the threshold for raw snapshots is increased. Conversely, when the total number of raw snapshots is high, the threshold is lowered to increase edge diversity.

Algorithm 1: LightMetis Partitioning Algorithm

Input: Sliding window size N_s , edge life cycle z , number of snapshots T , number of raw snapshots $|\mathcal{R}|$, set of raw snapshots $\mathcal{R}$

Output: Partitioning result PR

// Step 1: Compute Hierarchical Usage Counts

```

1  UG  $\leftarrow$  [min( $i, T - i, N_s$ ) for  $i \in \{1, \dots, T\}$ ] // array of size  $T$ 
2  U  $\leftarrow$  zeros( $T, |\mathcal{R}|$ ) // Initialize the usage count array
3  for  $i \leftarrow 1$  to  $T$  do
4    U[ $i, i : i + z$ ]  $\leftarrow$  UG[ $i$ ]
5  AR  $\leftarrow$  sum(U, axis = 1)
```

// Step 2: Execute the Lightweight Partitioning

```

6   $r_{min} \leftarrow N_s \cdot z \cdot (|\mathcal{R}| - 2z - 2N_s + 4) / |\mathcal{R}|$ 
7  snaps_for_partition  $\leftarrow$  []
8  for  $i \leftarrow 1$  to  $|\mathcal{R}|$  do
9    if AR[ $i$ ] >  $r_{min}$  then
10     snaps_for_partition.append( $\mathcal{R}_i$ )
11 all_edges  $\leftarrow$  vstack(snaps_for_partition)
12 sorted_edges  $\leftarrow$  sort(all_edges, axis = 1)
13 unique_edges, counts  $\leftarrow$  unique(sorted_edges, axis = 0)
14 PR  $\leftarrow$  Metis(unique_edges, counts)
15 return PR
```

We use Algorithm 1 to compute the usage count for each raw snapshot and perform the partitioning. First, we calculate the first level of $\mathcal{S}$ over $\mathcal{G}$ following the sliding window training to determine the usage count for each $\mathcal{G}_i$ (Line 1). Next, we apply second level of $\mathcal{G}$ over $\mathcal{R}$ following the edge life technique (Lines 2–4), substituting the usage count of $\mathcal{G}$ into the calculation for $\mathcal{R}$ to obtain

the final usage count of each $\mathcal{R}_j$. We initialize an array of size $T \times |\mathcal{R}|$ to store the usage counts of each $\mathcal{G}_i$ for corresponding $\mathcal{R}_j$ (Line 2) and place each $\mathcal{G}_i$ usage count in the appropriate position (Lines 3–4). Finally, we sum the usage counts for each $\mathcal{R}_j$ to compute its total usage count AR (Line 5).

In the partitioning stage, we first calculate the threshold for the minimum usage count of raw snapshots (Line 6), which is used to select only the raw snapshots with high usage frequency (Lines 7–10). These raw snapshots are aggregated into a new object, `all_edges` (Line 11) that is sorted to produce `sorted_edges` (Line 12). We then remove duplicate edges to create `unique_edges` and compute the frequency for each edge (Line 13). These unique edges and their frequencies form a weighted merged graph, which is fed into the Metis partitioning algorithm to reduce the communication overhead (Line 14). Example 3 illustrates the partitioning process.

EXAMPLE 3. In Figure 3, we set the sliding window size to $N_s = 2$ and the snapshot lifecycle size to $z = 3$. We compute the usage count of each snapshot per sliding window. Sliding window S_1 contains $\mathcal{G}_1$ and $\mathcal{G}_2$, while S_2 includes $\mathcal{G}_2$ and $\mathcal{G}_3$, resulting in a usage count table for the snapshots in the top-left corner. For the raw snapshots, $\mathcal{G}_1$ utilizes $\mathcal{R}_1, \mathcal{R}_2$, and $\mathcal{R}_3$, while $\mathcal{G}_2$ uses $\mathcal{R}_2, \mathcal{R}_3$, and $\mathcal{R}_4$. Hence, $\mathcal{R}_1$ is used once, and $\mathcal{R}_2$ is used three times (twice by $\mathcal{G}_2$), yielding the usage count table for the raw snapshots in the top-right corner. We set $r_{min} = 2 \times 6/8 = 1.5$, retaining raw snapshots with usage counts ≥ 1.5 , i.e., $\mathcal{R}_2$ through $\mathcal{R}_7$. After concatenating these snapshots and removing duplicates, we obtain a weighted merged graph, which is then processed using the Metis partitioning strategy.

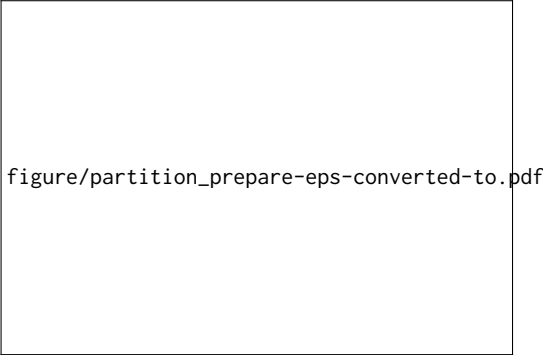


Figure 3: An example of partitioning.

5 Scheduling with Communication and Cache

5.1 Scheduling Framework Overview

To address communication and synchronization bottlenecks in distributed DGNN training, we propose a flexible framework that uses communication and cache scheduling. The amount of communication is dependent on three aspects: (i) the number of training epochs enabling communication; (ii) the number of sliding windows enabling communication per epoch; and (iii) the amount of communication per snapshot which is dependent on the partitioning strategy and graph characteristics. As full communication during training is time-consuming, our goal is to reduce the communication. Our framework includes three key hyperparameters.

DEFINITION 5 (FLEXIBLE COMMUNICATION HYPERPARAMETERS). In our framework, we implement flexible communication

reduction strategies. Specifically, we allow communication to occur once every k epochs, enable the transmission of s sliding windows per epoch, and define a communication ratio of p for each snapshot. Accordingly, we define the following parameters:

- **Communication Interval Size** (k): the number of epochs between consecutive communications;
- **Communication Window Number** (s): the number of sliding windows communicated within an epoch;
- **Communication Ratio** (p): the proportion of boundary embeddings communicated for each snapshot.

During DGNN training, we communicate only a fraction p of the boundary embeddings from s sliding windows when the condition for the communication interval k is satisfied. These embeddings are then cached for reuse during non-communication training iterations. This framework introduces several challenges:

- **Subgraph Reconstruction:** Sampling boundaries can incur significant costs in reconstructing subgraphs.
- **Accuracy and Convergence:** Dependence on cached embeddings may impact both accuracy and convergence negatively.
- **Hyperparameter Tuning:** Optimizing the hyperparameters k , s , and p can pose challenges.

To address these challenges, we present a merged boundary encoding technique, an empirical approach to hyperparameter settings, and a communication and cache scheduling strategy.

5.2 Communication and Cache Management

To facilitate communication management optimization, we define the notions of boundary vertex set and sampled boundary vertex set.

DEFINITION 6 (BOUNDARY VERTEX SET AND SAMPLED BOUNDARY VERTEX SET). Let $\mathcal{B}_i$ denote the **boundary vertex set** in the i^{th} graph snapshot of a dynamic graph. This set consists of vertices that are neighbors of local vertices but are not stored locally. These vertices must be retrieved through network communication when performing training on the current snapshot. To reduce communication overhead, we define the **sampled boundary set** $\check{\mathcal{B}}_i$ to be a subset of $\mathcal{B}_i$ utilized for efficient training.

In distributed DGNN training, once the boundary vertex set $\check{\mathcal{B}}_i$ has been obtained through sampling, the vertices must be re-encoded, extending the index range from N_{loc} (the number of local vertices) to $N_{loc} + |\check{\mathcal{B}}_i|$. After this re-encoding step, it is necessary to access the edge list and remove all edges whose source vertices are not in $\check{\mathcal{B}}_i$, while re-encoding edges where the source vertices do belong to the set. The process of matching source vertices and re-encoding edges is computationally expensive. Next, we define the set of embedding matrices that are needed in order to introduce the encoding and caching techniques.

DEFINITION 7 (EMBEDDING MATRIX GROUP). Let HM_l denote the **embedding matrix group** at layer l . This group consists of N_s matrices, each of size $|\cup_{i=1}^T \mathcal{B}_i| \times F_l$, where F_l is the embedding dimensionality of layer l , and $\mathcal{B}_i$ is the boundary vertex set of the i^{th} graph snapshot. The matrices in HM_l act as a sliding window shared cache, updated with embeddings received from communication. These embeddings are used to store and retrieve information for efficient computation during the training process.

Merged Boundary Encoding. Each snapshot stores the same local vertices, enabling the use of a single set of encodings. In contrast, the boundary vertices differ across snapshots, necessitating traditional encoding to be performed separately for the boundary vertices of each snapshot, along with the construction of a distinct HM embedding matrix for each. The conventional encoding technique prevents the sharing of boundary embeddings between snapshots and consumes considerable memory. To address this issue, we propose a merged boundary encoding that generates a unified set of encodings of the boundary vertices from all snapshots. Specifically, we consolidate the boundary sets from all snapshots to form $\mathcal{B} = \bigcup_{i=0}^{T-1} \mathcal{B}_i$ and subsequently encode $\mathcal{B}$. The resulting embedding matrix HM is stored persistently on the GPU. During each training iteration, we sample the boundary set for the i^{th} snapshot to obtain $\mathcal{B}_i$ and retrieve the embeddings of the vertices in $\mathcal{B}_i$ through communication. These embeddings are then transferred to the GPU, where they are mapped to the corresponding embedding matrix using a vertex map, and the relevant values are inserted. While this technique may result in some redundant boundary embeddings for each snapshot, the overall memory consumption remains relatively low when the partitioning strategy is effective at reducing the number of boundary vertices.

Sliding Window-Based Cache Sharing. We establish an embedding cache matrix for each snapshot within a sliding window, resulting in N_s embedding cache matrices, where N_s is the length of the sliding window. In each training iteration using the sliding window, boundary embeddings are updated and retrieved from the N_s matrices. Our cache sharing and update strategy focuses on sliding windows, rather than individual snapshots. Specifically, the embeddings updated in the i^{th} sliding window are written to the cache and shared with the j^{th} sliding window. Sharing within snapshots depends on their position in the sliding window. This technique recognizes that snapshots evolve over time and exhibit a certain degree of correlation; thus, sharing data from the i^{th} snapshot with the j^{th} snapshot is justified. Furthermore, snapshots that occupy the same position across different sliding windows undergo identical RNN operations. For instance, all snapshots at the i^{th} position in the sliding windows are derived from the previous $i - 1$ snapshots via RNN transformations, indicating that they have each undergone $i - 1$ RNN gate operations, thereby exhibiting larger similarity in their values.

figure/encode_and_cache-eps-converted-to.pdf

Figure 4: Communication and cache management.

Figure 4 shows the communication that integrates the merged boundary encoding and cache sharing. It shows the processes of wk_1 . Four snapshots are displayed, with boundary vertices categorized by their accessing worker. For instance, in snapshot $\mathcal{G}_1$, vertices v_1 and v_7 are requested by wk_2 , and vertex v_2 is requested

by wk_3 . We first take the union of the boundary sets across all snapshots, obtaining vertices v_1, v_7, v_2 , and v_5 , which are encoded with the local vertices. During training with a sliding window (size 2, indicated by purple, green, and red), boundary vertices are sampled and communicated, and their embeddings are inserted into the matrix according to the snapshot’s position in sliding window. Example 4 shows how to update and retrieve the cache.

EXAMPLE 4. *After communication in the first snapshot of the first sliding window, v_7 and v_2 (where h_4 is the encoded index corresponding to vertex v_7 , and h_5 corresponds to v_2) are inserted into the first embedding matrix. Similarly, v_5 from the second snapshot of the first sliding window is inserted into the second embedding matrix, h_6 . Vertex v_1 is not sampled in snapshot $\mathcal{G}_1$ of the first sliding window, and the embedding of v_1 , denoted as h_3 , is read from the first embedding matrix. Likewise, vertex v_5 is not sampled in snapshot $\mathcal{G}_2$ of the second sliding window, and the embedding of v_5 , i.e., h_6 , is retrieved from the second embedding matrix.*

5.3 Scheduling Strategy

We propose an online scheduling strategy that integrates the techniques discussed in Section 5.2. During distributed DGNN training, for every k epochs and across s sliding windows, we apply a merged boundary encoding communication strategy to sample a fraction p of the boundary vertices. These sampled vertices enable neighbor embedding retrieval, which is transferred to the GPU and inserted into the corresponding embedding matrices. In the remaining $k - 1$ non-communication epochs and $T - N_s + 1 - s$ non-communication sliding windows within communication epochs, the communication module is disabled, and boundary vertex embeddings are instead retrieved from local matrices. These embedding matrices are shared across all sliding windows. Thus, updates are preserved across windows and epochs, which improves convergence and accuracy. Example 5 illustrates the implementation of scheduling strategies for distributed DGNN training based on communication and caching.

figure/comm_cache_schedule-eps-converted-to.pdf

Figure 5: Scheduling process.

EXAMPLE 5. *In Figure 5, we use a sliding window length of 2 to exemplify our scheduling strategy. In the k^{th} epoch, we start by updating with the first sliding window, S_1 . We sample boundary vertices $\mathcal{B}_1$ and $\mathcal{B}_2$ from snapshots $\mathcal{G}_1$ and $\mathcal{G}_2$ in the first sliding window. Requests are sent to the workers hosting the vertices in $\mathcal{B}_1$ and $\mathcal{B}_2$ to retrieve their embeddings. These embeddings are then inserted into the HM matrix on the GPU. Specifically, the embedding of $\mathcal{B}_1$ is inserted into the first matrix, while the embedding of $\mathcal{B}_2$ is placed in the second matrix. These matrices are then concatenated with the local*

embedding matrices for subsequent training. During epochs $k+1$ to $2k-1$, we utilize these cached embedding matrices for training. In the $2k^{\text{th}}$ epoch, we communicate the second sliding window S_2 and write the embedding of $\mathcal{B}_2$ into the first matrix, as well as the embedding of $\mathcal{B}_3$ into the second matrix.

5.4 Hyperparameter Setting and Adjustment

We adjust three hyperparameters: the communication interval k , the number of communication windows s per epoch, and the communication ratio p . By setting appropriate values for k , s , and p , we balance communication volume and convergence. Additionally, we propose an adaptive strategy to reduce the risk of converging to a locally optimal loss.

Number of Communication Windows. It specifies the number of sliding windows during which communication is enabled in an epoch. Dynamic graphs evolve gradually and exhibit temporal correlations; thus, the overlap of boundary vertices between $\mathcal{G}_i$ and $\mathcal{G}_{i+1}$ typically larger than that between $\mathcal{G}_i$ and $\mathcal{G}_{i+2}$. Our objective is to distribute the s sliding windows as widely as possible to maximize boundary coverage. A larger sliding window size N_s allows more snapshots to be included in communication, reducing the required s . Conversely, a larger number of snapshots T necessitates increasing s to enhance coverage. Therefore, we define $s = \alpha \cdot T / N_s$.

Communication Ratio. It indicates the proportion of boundary vertices that are sampled in each snapshot of a sliding window during training. In our caching technique, data from each snapshot can be shared with all other snapshots. To calculate the boundary overlap between sliding windows—the proportion of shared boundaries across two windows—the boundary overlap between snapshots can be used instead. Given that snapshots derive from edge life, we approximate the snapshot coverage rate using the edge-life coverage, calculating the average coverage as $p_{\text{cover}} = N_{\text{lag}} / (N_{\text{lag}} + T)$. A high average coverage rate enables us to reduce the communication ratio p for improved efficiency, while a low coverage rate necessitates an increase in p to enhance convergence. Thus, we define the communication probability as $p = \beta \cdot (1 - p_{\text{cover}})$.

Communication Interval. It defines the frequency of communication, occurring every k epochs. The choice of the communication interval is influenced by the sliding window length and the number of snapshots. A longer sliding window allows for more communication opportunities, suggesting a larger interval to enhance training efficiency. Conversely, a higher number of snapshots suggests a shorter interval to reduce embedding cache staleness. Therefore, we express the communication interval as $k = \gamma \cdot N_s / T$.

Our analysis provides rules for hyperparameter tuning, and the introduction of coefficients α , β , and γ adds flexibility to the process.

Adaptive Communication Adjustment Strategy. To avoid local optima caused by insufficient communication, we propose an adaptive adjustment strategy. We define Δ_{loss} as the difference between the losses in consecutive iterations. If $\Delta_{\text{loss}} > 0$, the optimization is converging toward the optimal solution; otherwise, it indicates a divergence. When divergence occurs, we perform communication to explore the potential for escaping a local optimum. If the loss decreases after this adjustment, we terminate the communication; otherwise, we continue to perform communication, disregarding the established strategy.

Algorithm 2 presents the training process of SWASH. The training process involves the iterations of snapshots within sliding windows across epochs (Lines 1–3). Two modes exist for obtaining boundary vertex embeddings: communication mode (Lines 5–6) and caching mode (Line 8). The communication mode is employed only when the criteria for communication iterations and the communication window are satisfied or if the loss does not decrease compared to the previous iteration (Line 4); otherwise, we retrieve the required boundary embeddings from the cache. After obtaining boundary embeddings, they are concatenated with local embeddings to commence training (Lines 9–13), followed by an update of the current iteration’s loss reduction (Line 14). Upon reaching the communication iteration, we compute the set of communication windows for that iteration (Lines 15–16).

Algorithm 2: Sliding Window Training of SWASH

Input: Number of snapshots T , set of snapshots $\mathcal{G}$, window size N_s , communication ratio p , boundary node indices $\mathcal{B}$

Output: Trained model

```

1 for  $e \leftarrow 1$  to epoch do
2   for  $i \leftarrow 1$  to  $T - N_s + 1$  do
3     for  $t, \text{snap} \in \mathcal{G}[i : i + N_s]$  do
4       if  $e \% k = 0$  and  $c\_win.contains(i)$  or  $\Delta_{\text{loss}} \geq 0$  then
5         boundary_embs  $\leftarrow$  get_by_comm( $p, \mathcal{B}$ )
6         cache.boundary_embs[ $t$ ][ $\mathcal{B}$ ]  $\leftarrow$  boundary_embs
7       else
8         boundary_embs  $\leftarrow$  get_from_cache( $t$ )
9       embs  $\leftarrow$  concatenate(local_embs, boundary_embs)
10      loss_snap, h_state  $\leftarrow$  model(snap, h_state)
11      loss  $\leftarrow$  loss + loss_snap
12    loss  $\leftarrow$  loss /  $N_s$ 
13    loss.backward(); optimizer.step()
14     $\Delta_{\text{loss}} \leftarrow$  loss - losslst; losslst  $\leftarrow$  loss
15  if  $e \% k = 0$  then
16    c_win  $\leftarrow$  update_comm_win( $e, s, k, T - N_s + 1$ )

```

6 Theoretical Analysis

6.1 Cost Analysis

Partitioning Analysis. We analyze the time complexity of various partitioning strategies. Hash partitioning strategy assigns vertices to workers through traversal, resulting in a complexity of $O(N)$. The complexity of Metis-based partitioning strategy depends on the number of edges and workers, i.e., $O(|\mathcal{E}| \cdot \log m)$. SingleMetis randomly selects one snapshot for partitioning, yielding a cost of $O(\bar{E} \cdot N_{\text{lag}} \cdot \log m)$, where $\bar{E}$ is the average number of edges per snapshot and N_{lag} is the lifespan of each edge. AllMetis utilizes all snapshots, leading to a cost of $O(\bar{E} \cdot N_{\text{lag}} \cdot T \cdot \log m)$. LightMetis partitions the dynamic graph with a scaling ratio to filter important raw snapshots, resulting in a complexity of $O(p_r \cdot T \cdot \bar{E} \cdot \log m)$, where $0 < p_r < 1$.

Communication Cost Analysis. We analyze communication costs across three frameworks: full, partial, and flexible. In the full communication scenario, all vertices communicate during each epoch and sliding windows, resulting in a cost of $O(T_{\text{cvg}} \cdot S \cdot N_s \cdot E_{\text{rmt}})$,

where T_{cvg} is the total number of epochs, S is the number of sliding windows, N_s is the number of vertices, and E_{rmt} is communication overheads. In the partial communication framework, only a specified proportion of vertices communicate, leading to a reduced cost of $O(T_{\text{cvg}} \cdot S \cdot N_s \cdot p_c \cdot E_{\text{rmt}})$, where p_c is the proportion of communicating vertices. The flexible communication framework optimizes this by allowing communication every k epochs, enabling s sliding windows to communicate a proportion p of the vertices, resulting in a cost of $O(T_{\text{cvg}} \cdot s \cdot N_s \cdot p \cdot E_{\text{rmt}}/k)$. This technique enhances resource utilization while ensuring effective communication.

GPU Memory Cost Analysis. We evaluate the GPU memory costs associated with three caching strategies: L -hop caching, iteration caching, and sliding window caching. The L -hop cache necessitates storing L -hop neighbor vertices, resulting in a memory cost of $O(N^L \cdot F)$, where N is the number of vertices and F is the feature size. The iteration cache constructs a cache matrix for each snapshot, incurring an additional memory cost of $O(T \cdot L \cdot N_{\text{loc}} \cdot F)$, with T representing the total number of snapshots and N_{loc} denoting the number of locally cached vertices. In contrast, the sliding window caching allocates memory only for the sliding windows, leading to a cost of $O(N_s \cdot L \cdot N_{\text{loc}} \cdot F)$, where N_s is the number of sliding windows. This analysis highlights the superiority in memory consumption of SWASH among different caching techniques.

6.2 Error Analysis of Cache Utilization

We provide the error analysis between accurate and approximate aggregation results as follows.

Accurate Aggregation: The accurate embedding $z_{v,(t,s)}^\ell$ at iteration t and snapshot s is composed of local and communication embeddings, defined as:

$$z_{v,(t,s)}^\ell = \frac{1}{r} \sum_{i=1}^r h_{u_i,(t,s)}^\ell, \quad u_i \in \mathcal{N}(v) \quad (4)$$

Approximate Aggregation: The approximate embedding $\tilde{z}_{v,(t,s)}^\ell$ consists of local embeddings, and caching embeddings of iteration $\tilde{t}$ and snapshot $\tilde{s}$. Here, we assume iteration error $\mathbb{E}\|\Delta_{\text{itr}}\|_2^2 \leq \xi_{\text{itr}}^2$ and snapshot error $\mathbb{E}\|\Delta_{\text{snp}}\|_2^2 \leq \xi_{\text{snp}}^2$. And it is expressed as:

$$\begin{aligned} \tilde{z}_{v,(t,s)}^\ell &= \frac{1}{r} \left(\sum_{i=1}^{r_c} h_{u_i,(t,s)}^\ell + \sum_{i=r_c+1}^r h_{u_i,(\tilde{t},\tilde{s})}^\ell \right) \\ &= \frac{1}{r} \left(\sum_{i=1}^r h_{u_i,(t,s)}^\ell + \sum_{i=r_c+1}^r (\Delta_{\text{itr}}[u_i] + \Delta_{\text{snp}}[u_i]) \right) \end{aligned} \quad (5)$$

Interval Constraints: The upper limit of discrepancies in iterations and snapshots between cached and current accurate embeddings is expressed as:

$$\tilde{t} - t < \frac{k(T - N_s + 1)}{p}, \quad \tilde{w} - w < \frac{T}{ps} \quad (6)$$

Error Upper Bound: The error analysis between accurate and approximate aggregation is shown as:

$$\begin{aligned} \mathbb{E}\|z_{v,(t,s)}^\ell - \tilde{z}_{v,(t,s)}^\ell\|_2^2 &= \mathbb{E}\left\| \frac{1}{r} \sum_{i=r_c+1}^r (\Delta_{\text{itr}}(u_i) + \Delta_{\text{snp}}(u_i)) \right\|_2^2 \\ &\leq \frac{(r - r_c)^2}{r^2} (\mathbb{E}\|\Delta_{\text{itr}}(u_i)\|_2^2 + \mathbb{E}\|\Delta_{\text{snp}}(u_i)\|_2^2) \\ &\leq \frac{(r - r_c)^2}{r^2} \left(\frac{k^2(T - N_s + 1)^2}{p^2} \xi_{\text{itr}}^2 + \frac{T^2}{p^2 s^2} \xi_{\text{snp}}^2 \right) \end{aligned} \quad (7)$$

Table 2: Dynamic graph datasets (M=million).

Dataset	T	$ \mathcal{V} $	$ \mathcal{E} $	$ \mathcal{V}_{\text{total}} $	$ \mathcal{E}_{\text{el}} $	z
Slashdot	46	0.05M	0.14M	2.350M	2.81M	30
Stackexch	170	0.55M	1.30M	92.68M	33.6M	30
Ratings	59	2.15M	5.84M	126.6M	122M	30
Flickr	35	2.30M	33.1M	80.60M	218M	20
Youtube	35	3.22M	12.2M	112.8M	74.6M	10
Bitcoin	39	24.6M	122M	368.6M	842M	10

It is evident that increasing the number of local neighbors r_c through partitioning helps reduce errors. Additionally, decreasing the communication interval k , increasing the communication probability p , and increasing the number of communication sliding windows s will also contribute to error reduction. This flexibility enhances the communication operations of SWASH.

7 Experiments

7.1 Experimental Setup

DGNN Models. We evaluate two DGNN models: the Temporal Graph Convolutional Network (T-GCN) [48] and the Diffusion Convolutional Recurrent Neural Network (DCRNN) [21]. T-GCN integrates Graph Convolutional Networks (GCNs) with Gated Recurrent Units (GRUs) to effectively capture both spatial and temporal dynamics. DCRNN employs an encoder-decoder architecture to model temporal dependencies, utilizing bidirectional random walks on the graph to represent spatial relationships.

Datasets. We conduct experimental evaluations on six datasets, primarily focusing on regression tasks (RG) and link prediction tasks (LP). All datasets are publicly available on the website¹. The Slashdot (SLA), Flickr (FLI), YouTube (YOU), and Bitcoin (BIT) datasets represent social networks, where vertices model users, and edges represent relationships between them. The Ratings (RAT) dataset captures user-item interactions, with user ratings as the data points. The Stackexch (STA) dataset is a bipartite network from Stack Overflow, with nodes representing users and posts, and edges indicating when a user favorites a post. Table 2 provides an overview of the dataset statistics, where T is the number of snapshots, $|\mathcal{V}|$ is the number of vertices, $|\mathcal{E}|$ is the number of edges, $|\mathcal{V}_{\text{total}}|$ is the total number of unique vertices, $|\mathcal{E}_{\text{el}}|$ is the number of edges using the edge-life technique [3, 26, 33, 38], and z denotes the lifecycle.

Baselines. We compare our system SWASH against state-of-the-art distributed DGNN training systems and techniques, as follows:

- **PyGT-FS (PFS)** is an extension of PyGT [29], a distributed DGNN training system that utilizes standard vertex partitioning and operates in full snapshot training mode [3, 5, 29].
- **PyGT-VP (PVP)** builds upon PFS [29] by adopting sliding window training [10, 12, 38], which is preferred for DGNN training due to its efficiency and faster convergence.
- **BNS-DG (BNS)** incorporates the boundary sampling technique from BNS-GCN [37] for distributed DGNN training, transmitting only a subset of boundary vertices in each training iteration.
- **SANCUS (SCS)** employs the caching technique from SANCUS [27] originally designed for full-graph GNN training, adapting it for distributed DGNN training within the full snapshot training.

¹<https://networkrepository.com/dynamic.php>

Table 3: Overall comparison of regression tasks. *Bold* indicates the optimal time, while *underline* indicates the best loss. We show four metrics: iteration time (ST), convergence iterations (#CN), total convergence time (TT), and test loss or accuracy (TL).

System Model	Slashdot				Stackexch				Ratings				Flickr				Youtube				Bitcoin			
	ST	#CN	TT	TL	ST	#CN	TT	TL	ST	#CN	TT	TL	ST	#CN	TT	TL	ST	#CN	TT	TL	ST	#CN	TT	TL
PFS	4.35	20	86.98	0.449	27.3	111	3080	0.191	49.2	191	9405	0.391	44.4	111	4927	<u>0.500</u>	23.9	191	4567	0.261	OOM*			
PVP	2.09	78	163.1	0.416	3.19	119	386.8	0.189	21.6	141	3050	0.386	31.7	106	3360	0.500	18.1	236	4280	0.261	4.09	251	1026	0.374
BNS	0.62	87	53.88	<u>0.415</u>	0.82	119	99.48	0.189	1.89	99	539.4	0.386	5.52	106	585.1	0.500	4.23	191	808.8	0.261	1.56	201	314.2	0.375
SCS	1.49	20	29.71	0.449	8.47	111	947.2	0.191	17.7	161	2857	0.391	15.9	111	1763	0.500	8.70	191	1662	0.261	OOM*			
SWASH	0.28	60	16.44	0.422	0.42	118	39.85	<u>0.188</u>	0.56	155	91.61	<u>0.386</u>	0.74	106	78.05	0.500	0.49	196	96.57	<u>0.260</u>	1.13	211	239.3	<u>0.374</u>
PFS	0.92	191	175.5	<u>0.070</u>	6.87	181	1242	0.059	15.2	181	2742	0.147	14.6	351	5125	0.166	12.0	701	8425	0.051	3.81	191	726.8	0.180
PVP	0.55	181	99.45	0.073	0.99	171	169.2	0.061	6.64	211	1401	0.138	10.8	271	2925	0.166	6.03	991	5972	0.051	1.73	1191	2057	<u>0.119</u>
BNS	0.20	171	34.60	0.077	0.27	181	48.26	0.059	1.18	211	248.6	0.138	1.82	271	492.6	0.166	1.11	991	1097	0.051	0.87	1161	1012	0.120
SCS	0.54	191	97.52	0.074	3.59	181	651.5	0.059	16.4	211	769.5	0.138	9.84	351	3454	0.166	4.06	841	3414	0.051	2.51	681	1711	0.136
SWASH	0.10	181	18.41	0.073	0.10	181	18.76	<u>0.059</u>	0.20	211	43.13	<u>0.138</u>	0.24	271	65.96	<u>0.166</u>	0.26	991	258.0	<u>0.051</u>	0.60	1001	599.3	0.124

Table 4: Overall comparison of link prediction tasks.

System Model	Slashdot				Stackexch				Ratings				Flickr				Youtube				Bitcoin			
	ST	#CN	TT	TL	ST	#CN	TT	TL	ST	#CN	TT	TL	ST	#CN	TT	TL	ST	#CN	TT	TL	ST	#CN	TT	TL
PFS	2.09	30	62.87	96.78%	21.9	36	786.7	96.70%	49.4	36	1781	88.39%	45.0	11	495.4	93.29%	23.8	61	1450	93.57%	8.54	21	179.4	84.29%
PVP	2.12	30	63.46	<u>96.79%</u>	3.02	31	91.78	96.67%	21.3	20	417.3	88.99%	32.8	4	131.3	93.42%	18.0	31	557.1	93.52%	3.81	19	72.43	83.86%
BNS	0.70	15	10.53	96.46%	0.82	31	25.25	96.66%	3.54	20	70.01	89.16%	5.75	93	535.1	98.83%	3.2	26	83.59	93.83%	1.48	19	28.03	83.88%
SCS	1.76	20	35.26	96.69%	10.9	31	359.4	<u>96.76%</u>	23.1	26	550.5	88.65%	20.2	91	1835	94.73%	10.2	61	624.7	93.69%	5.48	21	115.0	84.30%
SWASH	0.29	14	4.191	96.41%	0.53	31	15.36	96.74%	1.70	20	26.08	89.11%	1.07	91	97.53	<u>98.87%</u>	0.88	26	22.91	<u>94.45%</u>	1.07	19	20.28	<u>84.30%</u>
PFS	0.95	81	77.12	51.70%	6.96	181	1255	60.61%	15.5	91	1403	68.19%	18.6	151	2810	57.9%	9.93	381	3785	50.6%	5.26	91	478.3	57.24%
PVP	0.55	41	23.07	51.87%	1.05	181	190.2	60.03%	7.94	91	606.7	68.20%	8.62	151	1302	57.9%	6.80	381	2590	50.6%	3.09	91	280.9	57.24%
BNS	0.22	41	9.709	51.87%	0.29	121	35.66	60.03%	0.86	91	78.23	68.20%	1.57	151	237.7	57.9%	1.33	381	506.8	50.6%	1.26	91	114.3	57.24%
SCS	0.56	31	18.25	51.58%	3.71	181	673.2	60.63%	6.67	91	722.2	68.18%	5.51	201	1107	57.9%	4.93	381	1879	50.6%	3.25	91	295.8	57.24%
SWASH	0.12	51	6.695	<u>51.88%</u>	0.12	121	14.56	<u>60.64%</u>	0.25	91	22.44	<u>68.22%</u>	0.23	151	34.85	<u>57.9%</u>	0.43	381	162.2	<u>50.6%</u>	0.98	91	89.61	<u>57.25%</u>

Some systems [3, 5, 10, 12, 33] employ frameworks that are not based on communication-based vertex partitioning, rendering them inherently incompatible with sliding window training; consequently, we exclude them from our comparisons. SWASH is open-sourced and the code is available ².

Environment Configuration and Hyperparameter Settings. We conduct tests on two environments. E_1 : It consists of five nodes, each equipped with 250 GB of DRAM, an Intel® Xeon® Silver 4110 CPU running at 2.10 GHz with 32 cores, and an NVIDIA RTX 2080 Ti with 11 GB of memory, with employing 10Gbps Ethernet connections. E_2 : It is a multi-GPU environment consisting of five NVIDIA RTX A6000 GPUs, each equipped with 48 GB of memory, paired with Intel® Xeon® Gold 6326 CPUs operating at 2.90 GHz. We split the dataset into training and testing sets with a 0.4/0.6 ratio and set the communication ratio—specifically, the boundary sampling ratio—for BNS to 10% to align with its initial configuration [37]. We provide a set of empirical hyperparameter configurations: $\alpha = 0.5$, $\beta = 0.2$, and $\gamma = 3$, to calculate the number of communication windows s , communication ratio p , and communication interval k , respectively. For our experiments, we use a two-layer T-GCN and a one-layer DCRNN, which are common configurations [10, 21, 26, 33, 48]. The sizes of the hidden layers for the six datasets are 16, 8, 8, 8, 8, and 4, respectively. Additionally, we set the length of sliding windows to 10 for all datasets. The Bitcoin dataset is run on environment E_2 as it exceeds the GPU memory of environment E_1 , while the other datasets are run on environment E_1 .

²<https://anonymous.4open.science/r/SWASH-Anonymous-0720>

7.2 Convergence and Efficiency Comparisons

Accuracy, Loss and Time Comparisons. We conduct extensive experiments on six datasets using the T-GCN and DCRNN models for both link prediction and regression tasks to validate the superiority of SWASH at Tables 3 and 4. For the regression task, we compare two metrics: mean squared error (lower is better) and time; for the link prediction task, we compare accuracy (higher is better) and time. The results indicate that SWASH achieves performance in terms of loss and accuracy comparable to the fully communication-based sliding window training system PVP, and even surpasses PVP in some instances. For example, with the T-GCN model, SWASH outperforms PVP by an average of approximately 3.15%. This improvement is primarily attributed to the randomness introduced by SWASH’s random communication vertices, which enhances generalization, while the adaptive communication adjustment strategy mitigates the impact of communication randomness on convergence. Notably, we observe that the BNS strategy, which utilizes random communication vertices, also outperforms the fully communication framework in some cases. For instance, in the Ratings dataset, BNS achieves an accuracy of 89.16% for the link prediction task, compared to 88.99% for PVP and 88.39% for PFS.

Furthermore, SWASH exhibits optimal convergence times, achieving acceleration factors of 1.21–102.7×, 1.35–44.34×, 1.28–8.38×, and 1.81–46.24× when compared to PFS, PVP, BNS, and SCS, respectively. The average acceleration factors are 46.77×, 19.82×, 4.03×, and 20.88× for these comparisons. This significant improvement is primarily attributed to its flexible communication framework

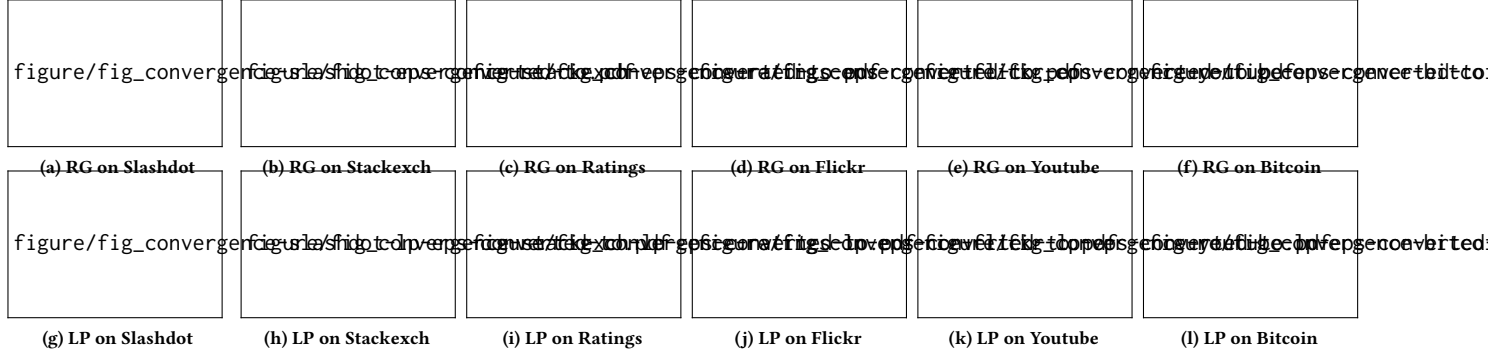


Figure 6: Convergence comparison.

and cache-sharing technique, which effectively reduce both the frequency and volume of communications. In terms of convergence time, the systems rank from best to worst as follows: SWASH, BNS, PVP (SCS), and PFS. This ranking arises because BNS notably minimizes communication time, while SCS reduces the overall number of communications. PVP’s sliding window training introduces a range of advantages compared to full-snapshot training. Conversely, PFS, being a fully communication-based and full snapshot training system, suffers from the most severe communication bottleneck, resulting in the lowest training efficiency.

Convergence Comparison. We present a convergence analysis that highlights the superiority of SWASH in terms of accuracy and efficiency across various models and tasks. Due to space constraints, our focus will primarily be on T-GCN for the subsequent analysis. As illustrated in Figure 6, SWASH achieves optimal convergence speed for both regression and link prediction tasks. For instance, in the regression task on the Flickr dataset using T-GCN, SWASH converges to below 0.6 in just 50.7 seconds, whereas BNS, SCS, PVP, and PFS require 238.6 seconds, 900.8 seconds, 1786.1 seconds, and 2261.2 seconds, respectively. This results in acceleration factors of 4.7 $\times$, 17.8 $\times$, 35.2 $\times$, and 44.6 $\times$, underscoring the effectiveness of SWASH. The rapid convergence of SWASH aligns with the earlier analysis regarding convergence time and accuracy. Specifically, the flexible communication framework and the lightweight partitioning strategy for sliding window training significantly reduce communication overhead. Meanwhile, using cached embeddings mitigates biases caused by data loss.

7.3 Comparison of Partitioning Strategies

We compare several partitioning strategies to evaluate the performance of LightMetis. The Hash partitioning distributes vertices evenly across workers using a round-robin strategy. SMetis applies Metis partitioning by randomly selecting the edges of a single snapshot. AMetis concatenates all snapshots and performs Metis partitioning on the combined data. We compare these strategies based on partitioning time, convergence time, and overall runtime. Although Hash incurs minimal partitioning time, it does not optimize communication, resulting in significantly longer convergence times during training. SMetis, which considers only one snapshot, also performs poorly in partitioning, leading to long convergence time. While AMetis achieves better partitioning quality, it requires considerably more partitioning time. In contrast, LightMetis achieves

partitioning quality comparable to AMetis, and with a substantial reduction in partitioning time.

For instance, as shown in Table 5 during training on the Ratings dataset, LightMetis experiences only a 9.6% decrease in partitioning performance compared to AMetis, while reducing partitioning time by 85.4%. This significant improvement is primarily due to the implementation of weighted reduction and pruning techniques that accelerate the partitioning process. Furthermore, we find that LightMetis outperforms AMetis in convergence time on the Stackexch dataset, achieving 150.71 seconds compared to AMetis’s 174.71 seconds. This advantage arises because AMetis concatenates all snapshots, making it optimal for full-snapshot training but less effective for sliding window training. Although Hash and SMetis have shorter partitioning times, their partitioning quality is inadequate; Hash employs a random partitioning strategy, while SMetis considers only the edges of a single snapshot.

Table 5: Comparison of partitioning strategies (sec).

		Hash	SMetis	AMetis	LightMetis
STA	Partitioning Time	0.2402	0.4104	107.19	15.670
	Convergence Time	207.96	399.68	174.71	150.71
	Total Time	208.20	400.09	389.09	166.38
RAT	Partitioning Time	0.7336	5.466	310.20	106.50
	Convergence Time	2097.4	1362.3	835.82	915.85
	Total Time	2098.1	1367.8	1146.0	1022.4

7.4 Speedup Test and Ablation Study

Additionally, we present comparative speedup in Figure 7 for all systems, offering a clearer demonstration of the advantages of SWASH over other systems. Specifically, SWASH achieves a speedup of 7.11–48.25 $\times$ in regression tasks for T-GCN, with an average iteration time yielding an overall speedup of 28.41 $\times$. This improvement is primarily attributed to our flexible communication framework and caching techniques. Furthermore, we observe that as the dataset size increases, SWASH generally attains greater speedups. For instance, it achieves speedups of 29.00 $\times$ on the Ratings dataset, 37.70 $\times$ on the Flickr dataset, and 48.25 $\times$ on the Youtube dataset. However, the speedup on the Bitcoin dataset exhibits a decline, primarily because, in a single-server multi-GPU environment, communication shifts to local communication, which reduces the benefits of communication optimization. Nevertheless, SWASH still achieves a speedup of 7.35 $\times$ compared to PFS on environment E_2 .

Furthermore, we present results from our ablation studies, which demonstrate that each individual optimization, as well as their combined effects, significantly contribute to the overall performance improvement. We establish the baseline as PVP and sequentially incorporate the following optimizations: LightMetis partitioning (+LaP), flexible communication ratio (+CP), flexible communication interval with caching (+CIC), and flexible communication window (+CWC). Each subsequent optimization builds upon the previous ones; for example, +CP integrates both the LightMetis optimization and the communication ratio optimization. As illustrated in Figure 8, each optimization positively impacts the performance of SWASH. Specifically, the individual optimizations yield speedups of 1.67–4.98 $\times$, 1.06–6.28 $\times$, 1.32–4.73 $\times$, and 1.12–2.94 $\times$, with average speedups of 3.00 $\times$, 3.50 $\times$, 2.04 $\times$, and 1.69 $\times$, respectively. Notably, the speedup associated with later optimizations tends to decrease, as the earlier partition and communication ratio optimizations have already mitigated some of the communication bottlenecks.

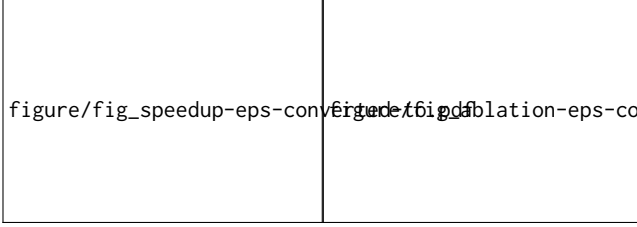


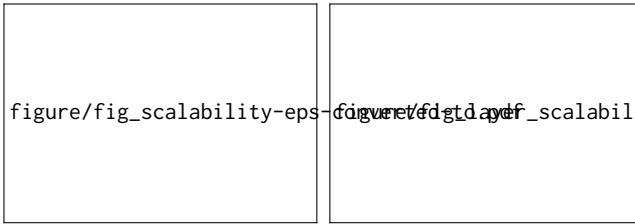
Figure 7: Speedup test.

Figure 8: Ablation study.

7.5 Scalability of Worker and Layer Numbers

We primarily test the scalabilities of worker number and layer number of SWASH on the Ratings dataset.

Worker Scalability. We evaluate worker scalability by varying the number of workers from 1 to 5 in Figure 9a. The results show that SWASH achieves near-linear scalability. For example, with 5 workers, BNS achieves a speedup of 3.02 $\times$, while SWASH achieves a 4.36 $\times$ speedup. This is due to the effective partitioning strategy, the controllable communication framework, and the use of caching in SWASH, which effectively mitigates communication bottlenecks. As the number of workers increases, SWASH continues to deliver excellent parallel speedup.



(a) Scalability of worker number.

(b) Scalability of layers.

Figure 9: Scalability test.

Layer Scalability. Next, we present the performance benefits across various layer configurations in Figure 9b. The results illustrate the training performance from a one-layer T-GCN to a four-layer T-GCN, demonstrating that SWASH consistently outperforms competing systems. Specifically, from layer 1 to layer 4, SWASH achieves average speedups of 2.42 $\times$, 4.18 $\times$, 3.67 $\times$, and 3.77 $\times$ compared to BNS, respectively. Notably, the speedups of SWASH relative

to the comparison baselines remain stable and do not diminish as the number of layers increases. This stability is attributed to the escalation of communication demands with an increasing number of layers; thus, the communication optimization techniques employed by SWASH continue to provide consistent benefits.

7.6 Breakdown and Hyperparameter Test

Breakdown. We provide a detailed breakdown of the runtime components on the Ratings dataset. *BDR*, *CPU*, *MSG*, and *TSF* correspond to the time required for sampling boundary vertices during communication, transferring embeddings from GPUs to CPUs for communication, constructing messages, and message passing, respectively. *GNN* and *RNN* represent the computation times for GNN and RNN operations. As shown in Table 6, the key advantage of SWASH lies in its significantly reduced communication overhead. For instance, SWASH requires a total of 41.53 seconds for communication-related tasks during training, compared to 123.76 seconds for BNS and 1481.93 seconds for PVP, which lacks communication optimizations. This improvement is largely attributed to SWASH’s flexible communication framework and partitioning strategy, which effectively minimizes both the frequency of communications and the volume of data transferred.

Table 6: Breakdowns.

System	Communication				Computation	
	<i>BDR</i>	<i>CPU</i>	<i>MSG</i>	<i>TSF</i>	<i>GNN</i>	<i>RNN</i>
PFS	N/A	206.1	1410	2866	22.4	4.99
PVP	N/A	63.03	456.5	962.4	7.18	1.60
BNS	21.96	10.98	26.29	64.53	7.64	1.60
SCS	N/A	106.7	656.4	1311	17.9	4.02
Ours	1.626	5.251	10.26	24.39	7.79	1.61

Table 7: HP test.

#P	<i>k</i>	<i>p</i>	<i>s</i>	Time
#1	13	0.1	1	34.28
#2	10	0.3	4	39.07
#3	7	0.5	7	48.03
#4	4	0.7	11	56.23
#5	1	1.0	14	98.57
Ours	9	0.13	3	31.99

Hyperparameter Setting. The key hyperparameters are α , β , and γ , corresponding to the communication window size s , the communication ratio p , and the communication interval k , respectively. Our hyperparameter tuning, therefore, focuses on optimizing s , p , and k . We evaluate five distinct parameter configurations, ranging from small to large values, as shown in Table 7. The results indicate that our selected configuration yields the best performance across different settings. While identifying an optimal parameter configuration is inherently difficult, the parameter calculation scheme we propose allows users to quickly derive empirical values for these three parameters, which have consistently demonstrated strong performance in our experiments.

8 Conclusion

Existing distributed DGNN systems struggle with communication bottlenecks in sliding window training due to inflexible communication and underutilized stale cache. We propose a flexible communication framework for sliding window training that enables partial boundary vertex communication in any epoch. Based on this, we design a lightweight partitioning strategy to reduce communication overhead and introduce a cache sharing technique to improve accuracy in non-communication iterations. We also present key designs for communication hyperparameters and propose an adaptive scheduling strategy to mitigate staleness effects. Experiments show that SWASH achieves an average speedup of 9.44 $\times$ over existing distributed DGNN systems while maintaining the accuracy of fully communicating, non-cached training.

References

- [1] Jiandong Bai, Jiawei Zhu, Yujiao Song, Ling Zhao, Zhixiang Hou, Ronghua Du, and Haifeng Li. 2021. A3T-GCN: Attention Temporal Graph Convolutional Network for Traffic Forecasting. *ISPRS Int. J. Geo Inf.* 10, 7, 485.
- [2] Kaidi Cao, Rui Deng, Shirley Wu, Edward W Huang, Karthik Subbian, and Jure Leskovec. 2023. Communication-Free Distributed GNN Training with Vertex Cut. *arXiv preprint arXiv:2308.03209* (2023).
- [3] Venkatesan T Chakaravathy, Shivmaran S Pandian, Saurabh Raj, Yogish Sabharwal, Toyotaro Suzumura, and Shashanka Ubaru. 2021. Efficient scaling of dynamic graph neural networks. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.
- [4] Chaoyi Chen, Dechao Gao, Yanfeng Zhang, Qiang Wang, Zhenbo Fu, Xuechang Zhang, Junhua Zhu, Yu Gu, and Ge Yu. 2023. NeutronStream: A Dynamic GNN Training Framework with Sliding Window for Graph Streams. *Proceedings of the VLDB Endowment* 17, 3, 455–468.
- [5] Fahao Chen, Peng Li, and Celimuge Wu. 2023. DGC: Training Dynamic Graphs with Spatio-Temporal Non-Uniformity Using Graph Partitioning by Chunks. *Proc. ACM Manag. Data* 1, 4 (2023).
- [6] Jinyin Chen, Xueke Wang, and Xuanheng Xu. 2022. GC-LSTM: graph convolution embedded LSTM for dynamic network link prediction. *Appl. Intell.* 52, 7 (2022), 7513–7528.
- [7] Songgaojun Deng, Huzefa Rangwala, and Yue Ning. 2019. Learning dynamic context graphs for predicting social events. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1007–1016.
- [8] Chi Thang Duong, Trung Dung Hoang, Hongzhi Yin, Matthias Weidlich, Quoc Viet Hung Nguyen, and Karl Aberer. 2021. Efficient streaming subgraph isomorphism with graph neural networks. *Proceedings of the VLDB Endowment* 14, 5 (2021), 730–742.
- [9] ZhengZhao Feng, Rui Wang, TianXing Wang, Mingli Song, Sai Wu, and Shuibing He. 2024. A Comprehensive Survey of Dynamic Graph Neural Networks: Models, Frameworks, Benchmarks, Experiments and Challenges. *arXiv preprint arXiv:2405.00476* (2024).
- [10] Kaihua Fu, Quan Chen, Yuzhuo Yang, Jiuchen Shi, Chao Li, and Minyi Guo. 2023. BLAD: Adaptive Load Balanced Scheduling and Operator Overlap Pipeline For Accelerating The Dynamic GNN Training. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–13.
- [11] Swapnil Gandhi and Anand Padmanabha Iyer. 2021. P3: Distributed deep graph learning at scale. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI})* 21, 551–568.
- [12] Mingyu Guan, Anand Padmanabha Iyer, and Taesoo Kim. 2022. DynaGraph: dynamic graph neural networks at scale. In *GRADES-NDA*. 6:1–6:10.
- [13] Xin Huang, Hong Cheng, Lu Qin, Wentao Tian, and Jeffrey Xu Yu. 2014. Querying k-truss community in large and dynamic graphs. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*. 1311–1322.
- [14] Ajay Kumar Jaiswal, Shiwei Liu, Tianlong Chen, Ying Ding, and Zhangyang Wang. 2023. Graph lading: Shockingly simple parallel gnn training without intermediate communication. In *International Conference on Machine Learning*. PMLR, 14679–14690.
- [15] George Karypis and Vipin Kumar. 1998. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on scientific Computing* 20, 1 (1998), 359–392.
- [16] Srijan Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks. In *SIGKDD*. 1269–1278.
- [17] Haoyang Li and Lei Chen. 2021. Cache-based GNN System for Dynamic Graphs. In *CIKM*. 937–946.
- [18] Haoyang Li and Lei Chen. 2023. EARLY: Efficient and Reliable Graph Neural Network for Dynamic Graphs. *Proc. ACM Manag. Data* 1, 2 (2023), 163:1–163:28.
- [19] Jia Li, Zhichao Han, Hong Cheng, Jiao Su, Pengyun Wang, Jianfeng Zhang, and Lujia Pan. 2019. Predicting Path Failure In Time-Evolving Graphs. In *SIGKDD*. ACM, 1279–1289.
- [20] Yiming Li, Yanyan Shen, Lei Chen, and Mingxuan Yuan. 2023. Orca: Scalable Temporal Graph Neural Network Training with Theoretical Guarantees. *Proc. ACM Manag. Data* 1, 1 (2023), 52:1–52:27.
- [21] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. 2018. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. In *ICLR*.
- [22] Antonio Longa, Veronica Lachi, Gabriele Santin, Monica Bianchini, Bruno Lepri, Pietro Lio, Franco Scarselli, and Andrea Passerini. 2023. Graph neural networks for temporal graphs: State of the art, open challenges, and opportunities. *arXiv preprint arXiv:2302.01018* (2023).
- [23] Vasmuddin Md, Sanchit Misra, Guixiang Ma, Ramanarayan Mohanty, Evangelos Georganas, Alexander Heinecke, Dhiraj Kalamkar, Nesreen K Ahmed, and Sasikanth Avancha. 2021. Distgtn: Scalable distributed training for large-scale graph neural networks. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14.
- [24] George Panagopoulos, Giannis Nikolentzos, and Michalis Vazirgiannis. 2021. Transfer graph neural networks for pandemic forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 4838–4845.
- [25] George Panagopoulos, Giannis Nikolentzos, and Michalis Vazirgiannis. 2021. Transfer Graph Neural Networks for Pandemic Forecasting. In *AAAI*. 4838–4845.
- [26] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao B. Schardl, and Charles E. Leiserson. 2020. EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs. In *AAAI*. 5363–5370.
- [27] Jingshu Peng, Zhao Chen, Yingxia Shao, Yanyan Shen, Lei Chen, and Jiannong Cao. 2022. SANCUS: Staleness-Aware Communication-Avoiding Full-Graph Decentralized Training in Large-Scale Graph Neural Networks. *Proc. VLDB Endow.* 15, 9 (2022), 1937–1950.
- [28] Xiao Qin, Nasrullah Sheikh, Chuan Lei, Berthold Reinwald, and Giacomo Domeniconi. 2023. SEIGN: A Simple and Efficient Graph Neural Network for Large Dynamic Graphs. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2850–2863.
- [29] Benedek Rozemberczki, Paul Scherer, Yixuan He, George Panagopoulos, Alexander Riedel, Maria Sinziana Astefanoaei, Oliver Kiss, Ferenc Bérés, Guzmán López, Nicolas Collignon, and Rik Sarkar. 2021. PyTorch Geometric Temporal: Spatiotemporal Signal Processing with Neural Machine Learning Models. In *CIKM*. 4564–4573.
- [30] Youngjoo Seo, Michaël Defferrard, Pierre Vandergheynst, and Xavier Bresson. 2018. Structured Sequence Modeling with Graph Convolutional Recurrent Networks. In *ICONIP*, Vol. 11301. 362–373.
- [31] Zhen Song, Yu Gu, Tianyi Li, Qing Sun, Yanfeng Zhang, Christian S. Jensen, and Ge Yu. 2023. ADGNN: Towards Scalable GNN Training with Aggregation-Difference Aware Sampling. *Proc. ACM Manag. Data* 1, 4, Article 229 (2023), 26 pages.
- [32] Zhen Song, Yu Gu, Jianzhong Qi, Zhigang Wang, and Ge Yu. 2022. EC-Graph: A Distributed Graph Neural Network System with Error-Compensated Compression. In *ICDE 2022*. 648–660.
- [33] Zhen Song, Yu Gu, Qing Sun, Tianyi Li, Yanfeng Zhang, Yushuai Li, Christian S Jensen, and Ge Yu. 2024. DynaHB: A Communication-Avoiding Asynchronous Distributed Framework with Hybrid Batches for Dynamic GNN Training. *Proceedings of the VLDB Endowment* 17, 11 (2024), 3388–3401.
- [34] Aymaz Taheri and Tanya Y. Berger-Wolf. 2019. Predictive temporal embedding of dynamic graphs. In *ASONAM*. ACM, 57–64.
- [35] John Thorpe, Yifan Qiao, Jonathan Eyolfson, Shen Teng, Guanzhou Hu, Zhihao Jia, Jinliang Wei, Keval Vora, Ravi Netravali, Miryung Kim, et al. 2021. Dorylus: Affordable, scalable, and accurate {GNN} training with distributed {CPU} servers and serverless threads. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. 495–514.
- [36] Jana Vatter, Ruben Mayer, and Hans-Arno Jacobsen. 2023. The evolution of distributed systems for graph neural networks and their origin in graph processing and deep learning: A survey. *Comput. Surveys* 56, 1 (2023), 1–37.
- [37] Cheng Wan, Youjie Li, Ang Li, Nam Sung Kim, and Yingyan Lin. 2022. Bns-gcn: Efficient full-graph training of graph convolutional networks with partition-parallelism and random boundary node sampling. *Proceedings of Machine Learning and Systems* 4 (2022), 673–693.
- [38] Chunyang Wang, Desen Sun, and Yuebin Bai. 2023. PiPAD: Pipelined and Parallel Dynamic GNN Training on GPUs. In *PPoPP*. 405–418.
- [39] Qiang Wang, Yanfeng Zhang, Hao Wang, Chaoyi Chen, Xiaodong Zhang, and Ge Yu. 2022. Neutronstar: distributed GNN training with hybrid dependency management. In *Proceedings of the 2022 International Conference on Management of Data*. 1301–1315.
- [40] Xuhong Wang, Ding Lyu, Mengjian Li, Yang Xia, Qi Yang, Xinwen Wang, Xinguang Wang, Ping Cui, Yupu Yang, Bowen Sun, et al. 2021. Apan: Asynchronous propagation attention network for real-time temporal graph embedding. In *Proceedings of the 2021 international conference on management of data*. 2628–2638.
- [41] Yanbang Wang, Yen-Yu Chang, Yunyu Liu, Jure Leskovec, and Pan Li. 2021. Inductive representation learning in temporal networks via causal anonymous walks. *arXiv preprint arXiv:2101.05974* (2021).
- [42] Yufeng Wang and Charith Mendis. 2023. TGOpt: Redundancy-Aware Optimizations for Temporal Graph Attention Networks. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 354–368.
- [43] Yaqi Xia, Zheng Zhang, Hulin Wang, Donglin Yang, Xiaobo Zhou, and Dazhao Cheng. 2023. Redundancy-free high-performance dynamic GNN training with hierarchical pipeline parallelism. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*. 17–30.
- [44] Da Xu, Chuanwei Ruan, Evren Körpeoglu, Sushant Kumar, and Kannan Achan. 2020. Inductive representation learning on temporal graphs. In *ICLR*.
- [45] Jiaxuan You, Tianyu Du, and Jure Leskovec. 2022. ROLAND: graph learning framework for dynamic graphs. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2358–2366.
- [46] Le Yu, Leilei Sun, Bowen Du, and Weifeng Lv. 2023. Towards Better Dynamic Graph Learning: New Architecture and Unified Library. *Advances in Neural Information Processing Systems* (2023).

- [47] Shuo Yu, Feng Xia, Shihao Li, Mingliang Hou, and Quan Z Sheng. 2023. Spatio-temporal graph learning for epidemic prediction. *ACM Transactions on Intelligent Systems and Technology* 14, 2 (2023), 1–25.
- [48] Ling Zhao, Yujiao Song, Chao Zhang, Yu Liu, Pu Wang, Tao Lin, Min Deng, and Haifeng Li. 2020. T-GCN: A Temporal Graph Convolutional Network for Traffic Prediction. *IEEE Trans. Intell. Transp. Syst.* 21, 9 (2020), 3848–3858.
- [49] Chuanpan Zheng, Xiaoliang Fan, Cheng Wang, and Jianzhong Qi. 2020. GMAN: A Graph Multi-Attention Network for Traffic Prediction. In *AAAI*. 1234–1241.
- [50] Yanping Zheng, Lu Yi, and Zhewei Wei. 2024. A survey of dynamic graph neural networks. *arXiv preprint arXiv:2404.18211* (2024).
- [51] Yuchen Zhong, Guangming Sheng, Tianzuo Qin, Minjie Wang, Quan Gan, and Chuan Wu. 2023. GNNFlow: A Distributed Framework for Continuous Temporal GNN Learning on Dynamic Graphs. *arXiv preprint arXiv:2311.17410*.
- [52] Hongkuan Zhou, Da Zheng, Israt Nisa, Vassilis N. Ioannidis, Xiang Song, and George Karypis. 2022. TGL: A General Framework for Temporal GNN Training on Billion-Scale Graphs. *Proc. VLDB Endow.* 15, 8 (2022), 1572–1580.
- [53] Hongkuan Zhou, Da Zheng, Xiang Song, George Karypis, and Viktor K. Prasanna. 2023. DistTGL: Distributed Memory-Based Temporal Graph Neural Network Training. In *SC*. 39:1–39:12.
- [54] Yifan Zhu, Fangpeng Cong, Dan Zhang, Wenwen Gong, Qika Lin, Wenzheng Feng, Yuxiao Dong, and Jie Tang. 2023. WinGNN: Dynamic Graph Neural Networks with Random Gradient Aggregation Window. In *SIGKDD*. 3650–3662.